

Generating Collider Events with MadGraph

A Complete, From-First-Principles Guide to Stage 1 of an
Agentic Particle-Physics Analysis Pipeline

HEP Agentic Pipeline — Stage 01 (Event Generation)
Pedagogical & supervisor-review document

June 4, 2026

Contents

How to read this guide	3
1 Why we are doing this	4
1.1 The problem: results that are “locked” to one model	4
1.2 How reinterpretation is done in practice	4
1.3 The concrete worked example	5
2 Physics background, from scratch	6
2.1 Collisions, events, and cross-sections	6
2.2 The Standard Model in one page	6
2.3 Why go beyond it: supersymmetry	6
2.4 The process we simulate	7
3 How MadGraph works	8
3.1 Step 1: Feynman diagrams and the matrix element	8
3.2 Step 2: from $ \mathcal{M} ^2$ to a cross-section — phase-space integration	8
3.3 Step 3: drawing events — unweighting	9
3.4 Step 4: writing the events — the LHE file	9
3.5 What MadGraph does <i>not</i> do here	9
4 The three cards	10
4.1 The process card — <i>what</i> to produce	10
4.2 The parameter card — the <i>physics point</i>	10
4.3 The run card — <i>how</i> to run	11
5 The computing environment, and why it matters	12
5.1 Why we cannot “just run it”	12
5.2 The solution: a self-contained environment	12
5.3 The generator itself	12
6 The reproduction, step by step	14
6.1 Provisioning (scripts 00–02)	14
6.2 Generating the process code (script 03, part D)	14
6.3 Installing the physics inputs (script 03, part E)	14
6.4 Generating events (script 03, part F)	15
7 Reading and validating the output	16
7.1 Anatomy of the LHE file	16
7.2 The cross-section and a sanity check	16
7.3 Why the inclusive total is larger: matching and merging	16
7.4 The event content	17

8	Generalising the workflow	18
9	Limitations and what comes next	19
A	Glossary	20
B	Reproducing this run end to end	22
C	References	23

How to read this guide

This document is the *pedagogical track* of the project. Its job is to let a reader *understand and audit* the event-generation stage, starting from very little assumed background — roughly a first-year undergraduate level in physics and programming. It is the counterpart to the *agent track* (`agent/WORKFLOW.md` and its nested files), which is a terse set of instructions for an automated agent to *execute* the same task. The two tracks are deliberately separate: here we explain *why*; there we say only *what to do*.

If you only want to *run* the workflow, you do not need this document — follow `agent/WORKFLOW.md`. If you want to know what every step means, whether the physics assumptions are sound, or how to teach this to someone new, read on. Cross-references to the agent files and to the change registry (`changes/`) appear throughout.

Boxes labelled **In one sentence** give a compressed takeaway; the surrounding text is the full explanation. Nothing here is required reading for the machine — it is for you.

Chapter 1

Why we are doing this

1.1 The problem: results that are “locked” to one model

The Large Hadron Collider (LHC) at CERN smashes protons together about 40 million times per second and records the debris. Physicists comb these collisions for signs of particles and forces beyond those we already know — collectively called *Beyond-the-Standard-Model* (BSM) physics. When an experiment such as ATLAS publishes a search, it almost always reports the result in terms of one or a few specific *benchmark models*: “if nature looked like *this*, we would (or would not) have seen it.”

The difficulty is that nature might look like something slightly different. A published exclusion “locks” the costly experimental work to the benchmark that happened to be chosen. The vast space of other plausible models is left untested, even though the same data often constrains them too. Re-using a published result to test a *new* model is called **reinterpretation**.

In one sentence. Reinterpretation asks: “given the data and tools the experiment already released, would this *other* model have been seen?”

1.2 How reinterpretation is done in practice

The reference for this project — Stark, Aristimuno Ots and Hance, “*Reduce, Reuse, Reinterpret: an end-to-end pipeline for recycling particle physics results*” (arXiv:2306.11055) — describes a software pipeline (**mapyde**) that chains together the same public tools the experiments use. To ask whether a new model is excluded, you:

1. **Generate** simulated collisions for the new model (this stage — MadGraph);
2. **Shower & hadronize** them into realistic sprays of particles (Pythia8);
3. **Simulate the detector’s** response (Delphes);
4. **Apply the published analysis** selection (SimpleAnalysis);
5. **Run the published statistical model** to get a limit (pyhf).

Figure 1.1 shows the chain. This guide covers **only stage 1**: turning a model and a process into simulated collisions. Everything after it consumes the file we produce.



Figure 1.1: The reinterpretation pipeline. The green box (MadGraph event generation) is the stage documented here; the arrows are the data files passed between stages.

1.3 The concrete worked example

Throughout, the running example is the signal sample at the heart of an ATLAS search for *sleptons* (defined in Chapter 2). The two input “cards” in this stage’s `inputs/` directory describe *slepton-pair production* at a specific mass point: sleptons of mass 200 GeV that each decay to a Standard-Model lepton plus an invisible neutralino of mass 150 GeV. The parameter card is byte-for-byte identical to the one ATLAS released on HEPData, so we are reproducing the actual signal simulation that underpins a published result.

Chapter 2

Physics background, from scratch

This chapter builds the minimum physics vocabulary needed to understand what we generate and why. A reader comfortable with cross-sections and supersymmetry can skim to Chapter 3.

2.1 Collisions, events, and cross-sections

At the LHC two beams of protons travel in opposite directions and cross inside detectors. When two protons (really, their constituent quarks and gluons — collectively *partons*) interact, we call the outcome an **event**: a list of the particles produced and their momenta.

How often does a given kind of event happen? That rate is encoded in a **cross-section**, σ , which has units of area. The standard unit is the *barn*; here we use femtobarns, $1 \text{ fb} = 10^{-39} \text{ cm}^2$. The number of events of a given type that an experiment collects is

$$N = \sigma \times \mathcal{L}, \tag{2.1}$$

where $\mathcal{L}$ is the **integrated luminosity** (a measured property of the data set, in inverse femtobarns). For example, the full LHC “Run 2” data set is $\mathcal{L} = 140 \text{ fb}^{-1}$; a process with $\sigma = 50 \text{ fb}$ therefore yields $N = 50 \times 140 = 7000$ events. Computing σ for a proposed model is exactly what stage 1 does, alongside producing example events.

In one sentence. The cross-section is the “how often” of a process; the events are simulated examples of “what it looks like.”

2.2 The Standard Model in one page

The **Standard Model** (SM) is our current best theory of fundamental particles. It contains: *quarks* (six kinds: up, down, charm, strange, top, bottom) and *leptons* (electron, muon, tau, and their neutrinos), which make up matter; and *force carriers* — the gluon (strong force), the photon and the W/Z bosons (electromagnetic and weak forces), and the Higgs boson (which gives mass). Protons are bound states of quarks and gluons, which is why a proton collision is really a collision of those constituents, with a *parton distribution function* (Chapter 3) describing how the proton’s momentum is shared among them.

2.3 Why go beyond it: supersymmetry

The SM is incomplete: it has no particle to explain the dark matter that astronomers observe, and it leaves some theoretical puzzles unresolved. **Supersymmetry** (SUSY) is a popular extension that pairs every SM particle with a heavier “superpartner.” Superpartners of the leptons are **sleptons** ($\tilde{\ell}$); the superpartner mixtures of the neutral bosons include the **neutralinos**

($\tilde{\chi}^0$). In many SUSY models the lightest superpartner is stable and neutral — a perfect dark-matter candidate — called the **Lightest Supersymmetric Particle** (LSP). Here the LSP is the lightest neutralino $\tilde{\chi}_1^0$.

A stable, neutral LSP does not interact in the detector: it escapes invisibly, carrying away momentum. Its signature is therefore **missing transverse momentum** (E_T^{miss}) — an apparent imbalance in the momenta we *do* see, in the plane transverse to the beams.

2.4 The process we simulate

Our running example is **direct slepton-pair production** followed by slepton decay:

$$pp \rightarrow \tilde{\ell}^+ \tilde{\ell}^-, \quad \tilde{\ell}^\pm \rightarrow \ell^\pm + \tilde{\chi}_1^0. \quad (2.2)$$

Two protons produce a slepton and an anti-slepton (via an intermediate photon or Z boson); each slepton decays to its Standard-Model partner lepton (e or μ) plus the invisible $\tilde{\chi}_1^0$. The experimental signature is therefore **two leptons + missing transverse momentum**.

The mass point “200/150”. A *simplified model* fixes just the few masses that matter and decouples everything else. Ours sets every slepton mass to 200 GeV and the neutralino LSP to 150 GeV; all coloured superpartners are given an enormous mass (4.5×10^9 GeV) so they never appear. The 50 GeV gap between slepton and neutralino is moderately “compressed”: the leptons come out fairly soft (low momentum), which is what the ATLAS search was optimised for. Each slepton is forced to decay to lepton + $\tilde{\chi}_1^0$ with 100% probability.

In one sentence. We are producing pairs of 200 GeV sleptons that each turn into a visible lepton plus an invisible 150 GeV dark-matter-like particle.

Chapter 3

How MadGraph works

MadGraph5_aMC@NLO (hereafter “MadGraph”) is a *matrix–element event generator*. This chapter explains what that means, in four conceptual steps. Tutorial slides referenced below are from the in–workspace deck *tutorial-CMSandATLAS-2016* (M. Zaro).

3.1 Step 1: Feynman diagrams and the matrix element

Quantum field theory computes the probability *amplitude* for a process as a sum of **Feynman diagrams** — pictorial shorthands for terms in a perturbative expansion. Each diagram encodes incoming and outgoing particles connected by *propagators* (internal virtual particles) at *vertices* (interaction points). The sum of all diagrams for a process is its amplitude $\mathcal{M}$. Probabilities depend on the *squared* amplitude $|\mathcal{M}|^2$.

MadGraph’s first job, given a model and a process, is to enumerate every contributing diagram and write computer code that evaluates $|\mathcal{M}|^2$ for any input momenta. For our example it generated **144 subprocesses with 1456 diagrams** (tutorial slides 20–21 show the analogous output for top–quark pairs).

3.2 Step 2: from $|\mathcal{M}|^2$ to a cross–section — phase–space integration

The cross–section is obtained by integrating $|\mathcal{M}|^2$ over all the ways the final–state particles can share the available energy and momentum — the **phase space** — and over the momentum fractions the colliding partons carry inside the protons. Schematically,

$$\sigma = \sum_{a,b} \int dx_a dx_b f_a(x_a, \mu) f_b(x_b, \mu) \int d\Phi |\mathcal{M}_{ab}|^2, \quad (3.1)$$

where $f_a(x, \mu)$ is the **parton distribution function** (PDF) — the probability of finding parton a carrying momentum fraction x at energy scale μ — and $d\Phi$ is the phase–space measure. This integral has many dimensions and is done numerically by **Monte Carlo integration**: throw random points, evaluate the integrand, average. MadGraph’s integrator is the *MadEvent* component (tutorial slides 14, 58).

PDFs. We use **nn231o1** (the NNPDF 2.3 leading–order set, LHAPDF id 247000), the same family ATLAS used. The PDF is built into MadGraph, so no extra software is needed.

3.3 Step 3: drawing events — unweighting

The Monte Carlo integration naturally produces *weighted* points: each random phase-space point comes with a probability weight. Real detector data, however, is “unweighted” — every recorded collision counts once. MadGraph therefore performs **unweighting**: it keeps or discards points in proportion to their weight, leaving a sample of equally-weighted events distributed exactly as nature would produce them. Each surviving event is a list of outgoing particles and their four-momenta.

3.4 Step 4: writing the events — the LHE file

The events are written in the **Les Houches Event** (LHE) format — a community-standard text file (here gzip-compressed). It has a header (`<init>`) with run metadata and the cross-section, an embedded copy of the input cards, and one `<event>` block per collision. This file is the deliverable of stage 1; Chapter 7 dissects it.

In one sentence. MadGraph turns “which model, which process” into “how often” (a cross-section) and “what it looks like” (a file of simulated events).

3.5 What MadGraph does *not* do here

At this stage the events are at **parton level**: the quarks and gluons have not yet been turned into the sprays of hadrons (“jets”) that a real detector sees, and there is no detector response. Those are the jobs of Pythia8 and Delphes downstream. MadGraph stops at the LHE file precisely because we did not install those tools (Chapter 5).

Chapter 4

The three cards

MadGraph is steered by three plain-text “cards” (tutorial slide 14). Two are provided; the third is generated automatically and then edited. The agent-track files `agent/checklists/*-decisions.md` give the terse “what to set” version; here we explain each.

4.1 The process card — *what* to produce

The process card is a short script of MadGraph commands. Ours (`inputs/proc_card.dat`) reads, in essence:

```
import model MSSM_SLHA2
define slep = el- el+ er- er+ mul- mul+ mur- mur+
generate p p > slep slep $ susystrong @1
add process p p > slep slep j $ susystrong @2
add process p p > slep slep j j $ susystrong @3
output -f -nojpeg
```

Line by line:

- `import model MSSM_SLHA2` — load the supersymmetric model (it ships with MadGraph).
- `define slep = ...` — a shorthand for “any of the left- or right-handed selectrons and smuons.” Multiparticle labels (tutorial slide 28) let one line stand for many.
- `generate p p > slep slep` — produce two sleptons from the proton–proton initial state. The `$ susystrong` clause *veto*es coloured superpartners as intermediate states, isolating the pure electroweak production we want.
- `add process ...j` and `...j j` — also allow one or two extra “jets” (partons) in the hard process. The `@1/@2/@3` tags label the 0-, 1- and 2-jet samples so they can be combined consistently later (Chapter 7).
- `output -f -nojpeg` — write the generated code to disk (`-f` overwrites, `-nojpeg` skips diagram images).

The only change we make is to name the output directory on the `output` line; this is purely organisational and is documented in `changes/DATA-AND-CARD-CHANGES.md` (change D1).

4.2 The parameter card — the *physics point*

The parameter card uses the **SUSY Les Houches Accord** (SLHA) format: blocks of numbers keyed by each particle’s *PDG code* (a standard integer ID). The important blocks for us:

- **Block MASS** — one line per particle, `<PDG> <mass> # name`. Ours sets PDG 1000011/2000011 (selectrons) and 1000013/2000013 (smuons) to 200 GeV, and PDG 1000022 (the neutralino LSP) to 150 GeV. Coloured superpartners get 4.5×10^9 GeV.
- **DECAY blocks** — the total width of each particle and its branching ratios. Each slepton has a single decay line at 100% to lepton + neutralino.

The provided card is identical to the ATLAS HEPData release. One technical adjustment was required to make MadGraph’s strict parser accept it; this is the subject of Section 6.3 and change D2.

4.3 The run card — *how to run*

MadGraph writes a sensible default run card when the process is output; we then edit a few fields. The ones that matter (full rationale in `changes/DATA-AND-CARD-CHANGES.md`, change D3):

Field	Value	Meaning
<code>ebeam1, ebeam2</code>	6500	beam energy in GeV (so $\sqrt{s} = 13$ TeV)
<code>pdlabel</code>	<code>nn23lo1</code>	the PDF set (NNPDF2.3LO)
<code>nevents</code>	1000	number of events to generate
<code>iseed</code>	0	random seed (0 = new each run)
<code>ickkw</code>	0	no MadGraph-side jet matching (see Ch. 7)
<code>xqcut</code>	0	matching cut (off, since <code>ickkw=0</code>)
<code>ptj</code>	20	minimum jet transverse momentum (GeV)

Chapter 5

The computing environment, and why it matters

This chapter is for readers who are new to scientific software. It explains every tool we installed and why. The terse instructions are in `agent/steps/02-...` and `03-...`; the formal change log is `changes/ENVIRONMENT-CHANGES.md`.

5.1 Why we cannot “just run it”

Scientific codes like MadGraph are sensitive to their environment. Three obstacles had to be solved on the machine used here:

1. **No Fortran compiler.** MadGraph’s numerical core is written in Fortran and must be *compiled* (translated to machine code) before it runs. The machine had no Fortran compiler.
2. **The wrong Python.** MadGraph 2.9.x is a Python program that uses two modules (`imp`, `distutils`) removed from Python 3.12 onward. The system had Python 3.13, which crashes it.
3. **No package manager** (no Homebrew/conda) and no administrator rights.

5.2 The solution: a self-contained environment

We installed **Miniforge**, a minimal distribution of the `conda` package manager, into the project’s `build/` directory — no admin rights, nothing system-wide. `conda` lets us create an isolated **environment** (named `mg5`) containing exactly the versions we need:

- **Python 3.10** — old enough to still have `imp/distutils`.
- **gfortran 13** — the GNU Fortran compiler, with the macOS SDK wired up so it can link executables.
- **make**, and the Python modules **six** and **numpy** that MadGraph needs.

This environment is regenerable from `scripts/00-...` and `01-...`, and lives under `build/` (which is git-ignored, because it is large and machine-specific).

5.3 The generator itself

We obtained MadGraph by cloning tag **v2.9.27** from the official GitHub repository. The reference paper used v2.9.3, but that exact version is not tagged on GitHub (tags start at v2.9.10). For a leading-order calculation like ours the patch level does not change the physics; this is argued in `changes/ENVIRONMENT-CHANGES.md` (E3). Four configuration lines were appended to make runs headless and reproducible (E4).

In one sentence. conda gives us a private, correctly-versioned compiler and Python so MadGraph builds and runs identically on any machine, with no admin rights.

Chapter 6

The reproduction, step by step

This chapter narrates exactly what happens when the workflow runs, including the errors that arise on a first attempt and *why* they occur — which is often where the real understanding lives.

6.1 Provisioning (scripts 00–02)

Running `scripts/00_install_miniforge.sh` downloads and installs Miniforge; `01...` creates the `mg5` environment and verifies the compiler by building a tiny Fortran “hello world” (a cheap way to retire the single most failure-prone part of the whole workflow); `02...` clones MadGraph and confirms it launches under Python 3.10.

First real error: the missing `six` module. On the first launch MadGraph stops with “*madgraph requires the six module.*” `six` is a small library that smooths over Python 2/3 differences; MadGraph imports it at start-up. The fix is one command: `conda install -n mg5 -c conda-forge six numpy`. (Recorded in `agent/checklists/troubleshooting.md`.)

6.2 Generating the process code (script 03, part D)

MadGraph reads the process card and writes the Fortran matrix-element code into `build/runs/slepton_200_1`. Crucially, it also **auto-generates** the default `run_card.dat` and `param_card.dat` at this moment — this is the “run card generated at runtime” that the workflow refers to. The console ends with “*Output to directory ... done.*” and reports 144 subprocesses / 1456 diagrams.

6.3 Installing the physics inputs (script 03, part E)

Second real error: `InvalidParam`. When we replace the default parameter card with the ATLAS one and launch, MadGraph aborts with `InvalidParam` and a Python traceback ending `int() ...'decay'`. The cause: the ATLAS card writes twelve Standard-Model decay lines in a non-standard form, e.g.

```
DECAY    6      decay 6 1.42E+00  # WT
```

The SLHA standard (and MadGraph’s parser) expects `DECAY 6 1.42E+00`; the extra `decay 6` tokens make the parser try to read the word “decay” as an integer. The fix (`scripts/normalize_param_card.py`) rewrites *only* those header lines to the standard form, preserving every width, code, and comment. No mass, mixing, or branching ratio changes — and the affected Standard-Model widths do not even enter our matrix element. This is change D2, documented at both the workflow and pedagogical levels.

Editing the run card. The four edits of Chapter 4 are applied. Among them, setting `ickkw=0` (and hence `xqcut=0`) is the physically meaningful one; the next section explains why.

Third real warning: `xqcut>0` but `ickkw=0`. On a first pass the auto-generated card had `ickkw=1` (turn on MadGraph’s own jet matching) with a non-zero `xqcut`. Because we intend to do the matching *later* in the parton shower instead (next chapter), the consistent choice is `ickkw=0` and `xqcut=0`; otherwise MadGraph warns that the combination is inconsistent. Setting `xqcut=0` removes the warning. This is change D3.

6.4 Generating events (script 03, part F)

`generate_events` compiles all subprocesses with `gfortran`, integrates the cross-section, unweights, and writes the LHE file. A benign message — “*the width of particle 13 ... too small for an s-channel resonance*” — refers to the muon’s tiny width; the muon is not an internal resonance in our process, so it is harmless. The run reports “*Cross-section : ...pb*” and “*1000 events*”.

Chapter 7

Reading and validating the output

A number is only meaningful if you can check it. This chapter shows how to read the LHE file and how to confirm the result is physically sensible. The terse version is `agent/steps/08-...`

7.1 Anatomy of the LHE file

Uncompressing the file and reading the `<init>` block reveals the run metadata and the cross-sections:

```
<init>
2212 2212 6.500000e+03 6.500000e+03 0 0 247000 247000 -4 3
4.72e-02 1.4e-04 1.04e-01 1      # 0-jet class:  ~47 fb
2.88e-02 1.7e-04 1.04e-01 2      # 1-jet class:  ~29 fb
2.83e-02 2.9e-04 1.04e-01 3      # 2-jet class:  ~28 fb
</init>
```

The first line says: proton (2212) on proton, each beam 6500 GeV, PDF id 247000, and three process classes. The next three lines give each class’s cross-section (in picobarns; 1 pb = 1000 fb). Each subsequent `<event>` block lists the particles: a status code (−1 incoming, +1 outgoing), the PDG id, and the four-momentum. In our sample every event contains exactly one $\tilde{\ell}^+\tilde{\ell}^-$ pair, and the slepton lines carry mass 200 GeV — a direct check that the parameter card took effect.

7.2 The cross-section and a sanity check

For one representative run the inclusive total was $\sigma \approx 104$ fb, split as $\sim 47/29/28$ fb across the 0-/1-/2-jet classes. Is that reasonable? ATLAS quotes (at higher order, NLO+NLL) $\sigma(\tilde{\ell}_L) = 43.9$ fb and $\sigma(\tilde{\ell}_R) = 16.7$ fb, totalling 60.6 fb. Higher-order calculations exceed leading order by a “ k -factor” the reference paper measured as 1.18, so the expected *leading-order* value is $60.6/1.18 \approx 51$ fb. Our **0-jet (Born)** piece, ≈ 47 fb, matches this to about 10% — exactly the agreement one expects at leading order.

7.3 Why the inclusive total is larger: matching and merging

Why is the inclusive total (104 fb) about double the Born value (47 fb)? Because we generated the 0-, 1- and 2-jet matrix elements *separately* and added them. There is overlap between them: a “0-jet” event whose parton later radiates looks like a “1-jet” event. Adding the classes naively **double-counts** this overlap.

The cure is **matching/merging**, and there are two schemes (tutorial slides 121, 143–146, 165–173):

- **MLM** matching lives inside MadGraph (`ickkw=1`, with a cut `xqcut`).
- **CKKW-L** merging lives in the parton shower (Pythia8), at a “merging scale.”

The ATLAS sample uses CKKW-L at a merging scale of $m(\tilde{\ell})/4 = 50\text{ GeV}$ — a step that happens in *stage 2*, not here. So at this stage we deliberately set `ickkw=0`: MadGraph emits the raw, overlapping multi-jet sample, and the downstream shower removes the double counting. The honest, comparable number from stage 1 alone is therefore the Born (0-jet) piece, which is why our sanity check used it. The extra-jet generation cut `ptj=20` (kept below the 50 GeV merging scale) regularises the otherwise-divergent extra-jet rates.

In one sentence. The inclusive total double-counts on purpose; the parton shower (stage 2) will merge the samples, and until then the Born piece is the trustworthy number.

7.4 The event content

A quick tally of the 1000 events shows the jet multiplicities split roughly 45/27/27% (0/1/2 jets), matching the cross-section fractions as it must for unweighted events. Left- and right-handed sleptons appear in about a 70:30 ratio, mirroring the ATLAS $\sigma(\tilde{\ell}_L) : \sigma(\tilde{\ell}_R)$ ratio. These internal consistencies are themselves validation.

Chapter 8

Generalising the workflow

The point of the project is not this one sample but a *repeatable process*. The agent track is written to be general; here is the decision logic a human (or agent) follows for a *new* request, mirroring `agent/checklists/`.

- 1. What process?** Translate the request into `generate/add process` lines: choose the model, the final state, whether to include extra jets, and which intermediate states to veto. (`checklists/process-card-decisions.md`.)
- 2. What physics point?** Edit the `Block MASS` entries for the relevant PDG codes and set the decays; decouple anything that should not appear. Normalise the SLHA card if MadGraph rejects it. (`checklists/parameter-card-decisions.md`.)
- 3. How to run?** Set $\sqrt{s}$ via `ebeam`, the event count, the seed, and — if the process has multiple jet multiplicities — the matching choice. For shower-side (CKKW-L) merging use `ickkw=0`; for self-contained MLM use `ickkw=1` with an `xqcut`. (`checklists/run-card-decisions.md`.)
- 4. Check in.** Confirm the process card and the key run settings with the scientist before a long run; incorporate feedback by revisiting only the affected step. (`shared/agent/ORCHESTRATION.md`.)
The same scripts work for any leading-order process; only the cards and a few run-card fields change. A different mass point reuses the already-compiled process directory and only re-runs the integration.

Chapter 9

Limitations and what comes next

- **Parton level only.** The output has no parton shower, hadronisation, or detector effects. It is an *input* to those stages, not a physics result on its own.
- **Leading order.** Cross-sections are LO; a k -factor (here 1.18) is applied downstream to approximate higher orders.
- **No merging yet.** The inclusive multi-jet total double-counts until the stage-2 shower merges it (Section 7.3).
- **Version.** v2.9.27, not the paper's v2.9.3 (unavailable on GitHub); immaterial at LO.
- **Statistics.** The example uses 1000 events for speed; production samples use far more (the paper used 50000). Raise `nevents` and enable multi-core running for those.

The next stage (Pythia8) consumes this LHE file, showers and hadronises it, and performs the CKKW-L merging; see `../../../../shared/agent/ORCHESTRATION.md`.

Appendix A

Glossary

cross--section (σ)

interaction rate, in units of area; $\text{events} = \sigma \times \mathcal{L}$.

luminosity ($\mathcal{L}$)

a measure of how much data was collected, in fb^{-1} .

event

one simulated (or real) collision: a list of particles and momenta.

matrix element

the quantum amplitude $\mathcal{M}$; $|\mathcal{M}|^2$ drives probabilities.

phase space

the set of allowed final-state momentum configurations.

PDF

parton distribution function; the momentum content of the proton.

LHE

Les Houches Event format; the parton-level event file MadGraph writes.

parton level

before the shower turns quarks/gluons into jets of hadrons.

LO / Born

leading order; the lowest-order (here 0-jet) contribution.

k --factor

ratio of a higher-order to the LO cross-section.

MLM / CKKW--L

two schemes for combining matrix-element multiplicities with the shower without double counting.

SLHA

SUSY Les Houches Accord; the parameter-card text format.

LSP / $\tilde{\chi}_1^0$

lightest supersymmetric particle; stable, invisible $\Rightarrow$ missing momentum.

slepton ($\tilde{\ell}$)

scalar superpartner of a lepton.

PDG code

standard integer identifier for each particle.

Appendix B

Reproducing this run end to end

From the stage directory (`stages/01-event-generation/`):

```
bash scripts/00_install_miniforge.sh
bash scripts/01_create_env.sh
build/tools/miniforge3/bin/conda install -y -n mg5 -c conda-forge six numpy
bash scripts/02_get_madgraph.sh
NEVENTS=1000 bash scripts/03_run_madgraph.sh run_02
```

Output: `build/runs/slepton_200_150/Events/run_02/unweighted_events.lhe.gz`.

Appendix C

References

1. G. Stark, C. Aristimuno Ots, M. Hance, *Reduce, Reuse, Reinterpret: an end-to-end pipeline for recycling particle physics results*, arXiv:2306.11055.
2. J. Alwall *et al.*, *The automated computation of tree-level and next-to-leading order differential cross sections, and their matching to parton shower simulations*, JHEP 07 (2014) 079 (MadGraph5_aMC@NLO).
3. M. Zaro, *MadGraph5_aMC@NLO tutorial* (CMS & ATLAS, 2016) — the slide deck in this workspace.
4. ATLAS Collaboration, *Searches for electroweak production of supersymmetric particles with compressed mass spectra in $\sqrt{s} = 13$ TeV pp collisions*, Phys. Rev. D 101 (2020) 052005, and its HEPData record.
5. The agent track of this stage (`agent/WORKFLOW.md`) and the change registry (`changes/`).